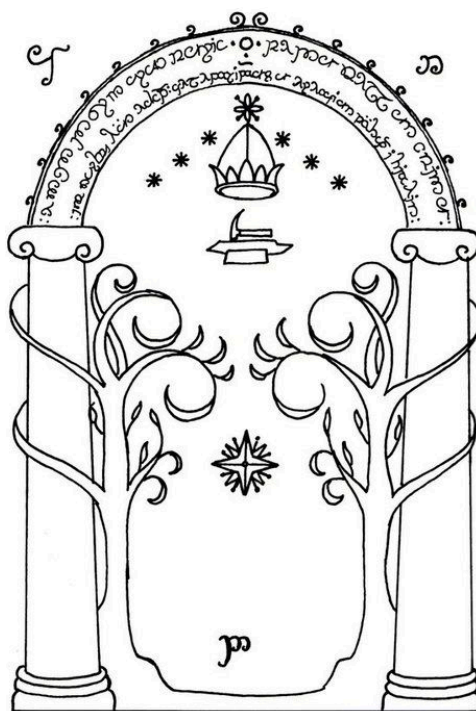




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

4 de septiembre de 2026

Indice:

- [1. Equipo](#)
- [2. Repositorio](#)
- [3. Dominio](#)
- [4. Construcciones](#)
- [5. Casos de prueba](#)
- [6. Ejemplos](#)

1. Equipo

Nombre	Apellido	Legajo	E-mail
Sofia Belen	Mazzini	65253	smazzini@itba.edu.ar
Alan Gabriel	Navarro	63330	anavarro@itba.edu.ar
Santino	Giambelluca	64697	sgiambelluca@itba.edu.ar

Ignacio	Ferrero Puertolas	64399	iferreropuertolas@itba.edu.ar
---------	-------------------	-------	--

2. Repositorio

La solución será versionada en: <https://github.com/sgiambelluca/3d-render-TLA>

3. Dominio

El lenguaje a desarrollar es un DSL (Domain Specific Language). El dominio seleccionado abarca la descripción declarativa de escenas tridimensionales (cámara, mallas geométricas, materiales y fuentes de luz) y su compilación hacia un artefacto ejecutable que produce la representación bidimensional (imagen) de dicha escena mediante técnicas de *software rendering*.

El lenguaje utiliza un paradigma declarativo basado en bloques anidados delimitados por llaves, donde cada bloque define una entidad de la escena (camera, light, material, mesh) mediante pares clave-valor tipados. El backend del compilador procesa estas definiciones y genera un documento HTML con JavaScript embebido, que al ejecutarse sobre un *runtime* de rasterización propio (implementado una única vez sobre la API de <canvas> del navegador), produce la imagen final de la escena. Este enfoque evita cualquier dependencia de aceleración gráfica por hardware, resultando en un artefacto de salida autocontenido y ejecutable en cualquier navegador sin instalación adicional.

4. Construcciones

(I). Jerarquía por bloques anidados: El lenguaje delimita sus scopes mediante llaves (`{ }`), donde una escena (`scene`) contiene bloques de entidades (`camera`, `light`, `material`, `mesh`, `render`), y cada entidad se define mediante pares clave-valor.

(II). Definición de Cámara (`camera`): Construcción única por escena para instanciar el punto de vista desde el cual se renderiza, con atributos de posición (`pos`), punto de mira (`look_at`) y campo de visión (`fov`). El valor de `fov` debe ser un ángulo estrictamente mayor a `0deg` y estrictamente menor a `180deg`.

(III). Definición de Luces (`light`): Bloques para instanciar fuentes lumínicas de dos subtipos (`directional`, con vector de dirección no nulo, y `point`, con posición) cada una con color e intensidad (`intensity`), siendo esta última un valor numérico en el rango `[0, 1]`.

(IV). Definición de Materiales (`material`): Construcciones para declarar propiedades de apariencia (color, rugosidad especular mediante `shininess`) reutilizables por múltiples mallas. El valor de `shininess` es un entero no negativo, donde `0` representa un material completamente mate.

(V). Definición de Mallas Geométricas (`mesh`): Bloques que instancian primitivas geométricas de tres subtipos (`box`, `sphere`, `plane`) cada una con sus parámetros dimensionales específicos y una

referencia a un material previamente declarado.

(VI). Bloque de Renderizado (render): Construcción que especifica la resolución de la imagen de salida y el nombre del archivo a generar.

(VII). Tipos de datos: Cadenas de texto ("..."), numéricos (enteros y de punto flotante) ,ángulos con sufijo (60deg), colores hexadecimales (#rrggbb), vectores de 3 componentes ((x, y, z)), e identificadores.

(VIII). Referencias entre entidades: El lenguaje permite que una entidad referencie a otra previamente declarada dentro de la misma escena (ej. un mesh referencia un material por su identificador), lo cual requiere resolución en la etapa de análisis semántico.

(IX). Comentarios: De línea simple utilizando //.

(*) Nota sobre nomenclatura: a lo largo de la especificación, pos se utiliza como abreviatura del atributo position, que representa la posición de una entidad en el espacio tridimensional. Aplica tanto a camera como a light de tipo point y a mesh.

5. Casos de prueba

5.1. Casos de aceptación

Se proponen los siguientes casos iniciales de prueba de aceptación:

- (I).** Un programa que defina una escena mínima, compuesta únicamente por una camera y un bloque render, sin ninguna malla ni luz declarada.
- (II).** Un programa que defina una escena completa: una camera, una light de tipo directional, un material, y un mesh de tipo box que referencia dicho material correctamente.
- (III).** Un programa que declare dos o más bloques mesh distintos referenciando el mismo material.
- (IV).** Un programa que defina una light de tipo point, utilizando el atributo pos (position) en lugar de direction.
- (V).** Un programa que declare múltiples bloques light dentro de la misma escena.
- (VI).** Un programa que defina un mesh de tipo sphere, utilizando el atributo radius en lugar de size.
- (VII).** Un programa que defina un mesh de tipo plane, representando el piso de la escena.
- (VIII).** Un programa que declare una camera con un valor de fov cercano al límite superior permitido (179°).
- (IX).** Un programa que declare un material con shininess igual a 0 (material completamente mate).
- (X).** Un programa que configure un bloque render con una resolución rectangular no cuadrada (ej. 1920x1080).

5.2 Casos de prueba de rechazo:

- (I). Un programa en el que un mesh referencie, mediante su atributo material, un identificador que no fue declarado en ningún bloque material de la escena.
- (II). Dos bloques material (o cualquier entidad) no pueden compartir el mismo identificador dentro de la misma scene.
- (III). Un programa en el que una light de tipo directional declare un vector direction igual a (0,0,0), no normalizable.
- (IV). Un programa en el que una camera declare un valor de fov fuera del rango físicamente válido (ej. 200°).
- (V). Un programa en el que una light declare un valor de intensity fuera del rango permitido (ej. -0.5 o 1.5).

6. Ejemplos

La escena más simple posible aceptada por el lenguaje: solo una cámara y el bloque de renderizado, sin ninguna malla ni luz.

```
scene "vacía" {
  camera main {
    pos: (0, 2, 5)
    look_at: (0, 0, 0)
    fov: 60deg
  }

  render {
    resolution: (800, 600)
    output: "frame.html"
  }
}
```

Cód. 6.1: Escena mínima, compuesta únicamente por una cámara y el bloque de renderizado.

Definición de cámara para observar una esfera con material propio y acompañado por una emisión de luz, donde devuelve un HTML con resolución fija para poder observar dicha escena:

```
scene "esfera_lampara" {
  camera lateral {
    pos: (3, 2, 4)
    look_at: (0, 0, 0)
    fov: 50deg
  }
  light lampara : point {
    pos: (0, 4, 0)
    color: #88aaff
    intensity: 0.8
  }
  material azul {
```

```

    color: #2e86de
    shininess: 64
  }
  mesh pelota : sphere {
    pos: (0, 0.5, 0)
    radius: 0.7
    material: azul
  }
  render {
    resolution: (640, 480)
    output: "esfera.html"
  }
}

```

Cód. 6.2: Escena con una esfera de material propio iluminada por una fuente de luz puntual.